ML TASK

TASK-1

A researcher wants to predict the life expectancy of countries based on socio-economic and health factors. You are tasked with building a Linear Regression model that can predict life expectancy using historical data.

CODE:

```
import kagglehub
import pandas as pd
import os

path = kagglehub.dataset_download("kumarajarshi/life-expectancy-who")
print("Dataset downloaded to:", path)
print("Files in folder:", os.listdir(path))

df = pd.read_csv(os.path.join(path, "Life Expectancy Data.csv"))
df.head()
```

DATASET:

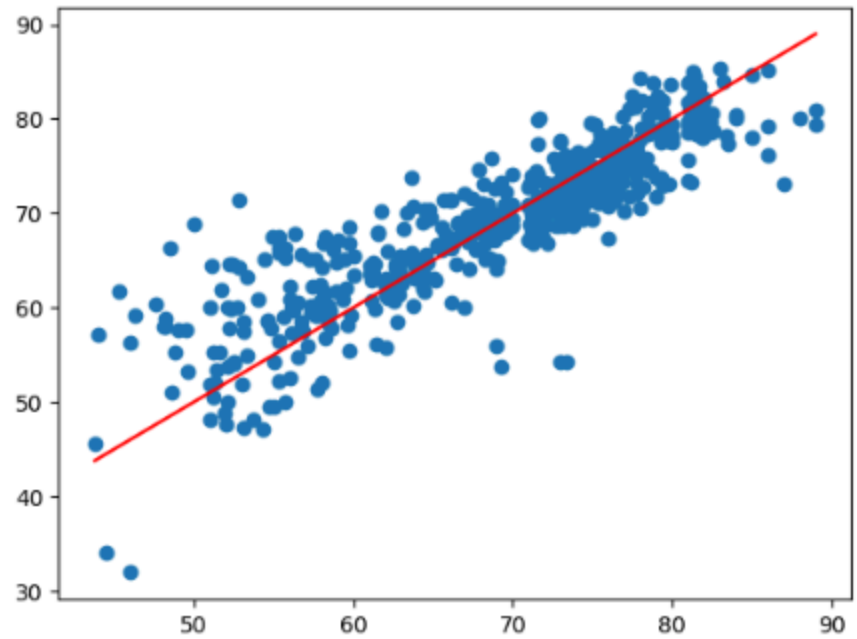
	Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure	Hepatitis B	Measles	...	Polio	Total expenditure	Diphtheria	HIV/AIDS	GDP
0	Afghanistan	2015	Developing	65.0	263.0	62	0.01	71.279624	65.0	1154	...	6.0	8.16	65.0	0.1	584.259210
1	Afghanistan	2014	Developing	59.9	271.0	64	0.01	73.523582	62.0	492	...	58.0	8.18	62.0	0.1	612.696514
2	Afghanistan	2013	Developing	59.9	268.0	66	0.01	73.219243	64.0	430	...	62.0	8.13	64.0	0.1	631.744976
3	Afghanistan	2012	Developing	59.5	272.0	69	0.01	78.184215	67.0	2787	...	67.0	8.52	67.0	0.1	669.959000
4	Afghanistan	2011	Developing	59.2	275.0	71	0.01	7.097109	68.0	3013	...	68.0	7.87	68.0	0.1	63.537231

```
import kagglehub
import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
path = kagglehub.dataset_download("kumarajarshi/life-expectancy-who")
df = pd.read_csv(os.path.join(path, "Life Expectancy Data.csv"))
df.columns = df.columns.str.strip()
```

```
df.fillna(df.mean(numeric_only=True), inplace=True)
X = df[['Adult Mortality', 'Alcohol', 'GDP', 'Schooling', 'HIV/AIDS']]
y = df['Life expectancy']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(mean_squared_error(y_test, y_pred))
print(r2_score(y_test, y_pred))
print(pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
}))

plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color='red')
plt.show()
```

OUTPUT:

```
20.735381733238206
0.7606589085483256
           Feature  Coefficient
0  Adult Mortality    -0.025286
1          Alcohol     0.163961
2              GDP     0.000066
3        Schooling     1.277675
4         HIV/AIDS    -0.502265
```



TASK-2:

A real estate analyst wants to predict the **median value of houses** in different neighborhoods based on various property and neighborhood features. You are tasked with building a **Multiple Linear Regression model** that can predict housing prices using numerical predictors.

CODE:

```
import kagglehub

import pandas as pd

import os

path = kagglehub.dataset_download("prateekmaj21/boston-housing-dataset")

print("Dataset downloaded to:", path)
```

```
print("Files in folder:", os.listdir(path))

df = pd.read_csv(os.path.join(path, "boston.csv"))

df.head()
```

DATASET:

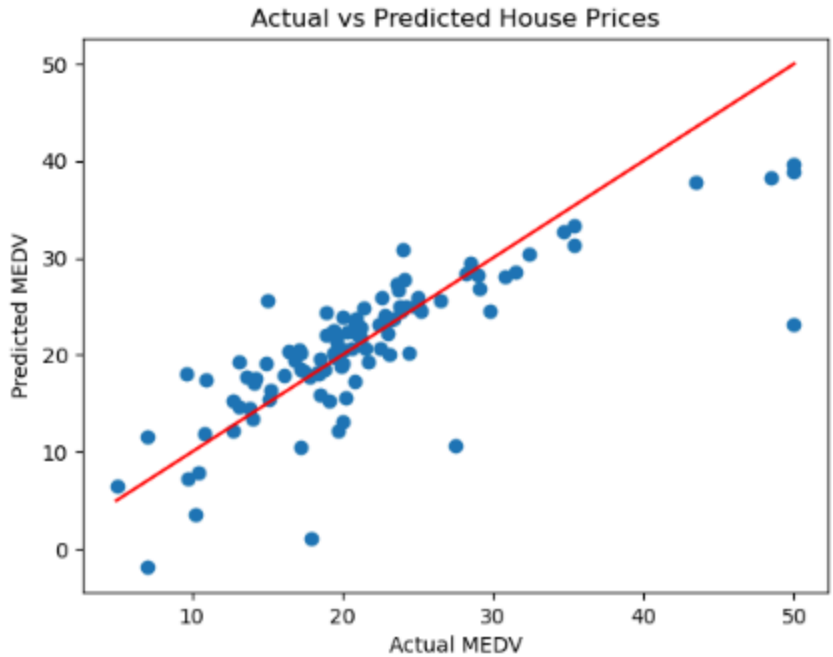
	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV	BIAS_COL
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.0	1
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.6	1
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7	1
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4	1
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.2	1

```
import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

df.fillna(df.mean(numeric_only=True), inplace=True)

X = df[['RM', 'LSTAT', 'PTRATIO', 'INDUS', 'NOX', 'AGE']]

y = df['MEDV']

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

```
print("R² Score:", r2_score(y_test, y_pred))
print("\nCoefficients:")
print(pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
}))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual MEDV")
plt.ylabel("Predicted MEDV")
plt.title("Actual vs Predicted House Prices")
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color='red')
plt.show()
```

OUTPUT:

```
Mean Squared Error: 26.782523432699975
R² Score: 0.6347860115964863

Coefficients:
   Feature  Coefficient
0       RM     4.776817
1    LSTAT    -0.593496
2  PTRATIO    -0.948690
3    INDUS     0.065783
4      NOX    -8.650123
5      AGE     0.032273
```



TASK-3:

A car manufacturer wants to predict the fuel efficiency (miles per gallon, MPG) of its vehicles based on engine size, weight, and horsepower. You are tasked with building a Polynomial Regression model to capture non-linear relationships between features and MPG.

CODE:

```
import kagglehub

import pandas as pd

import os

path = kagglehub.dataset_download("uciml/autompg-dataset")

print("Dataset downloaded to:", path)

print("Files:", os.listdir(path))
```

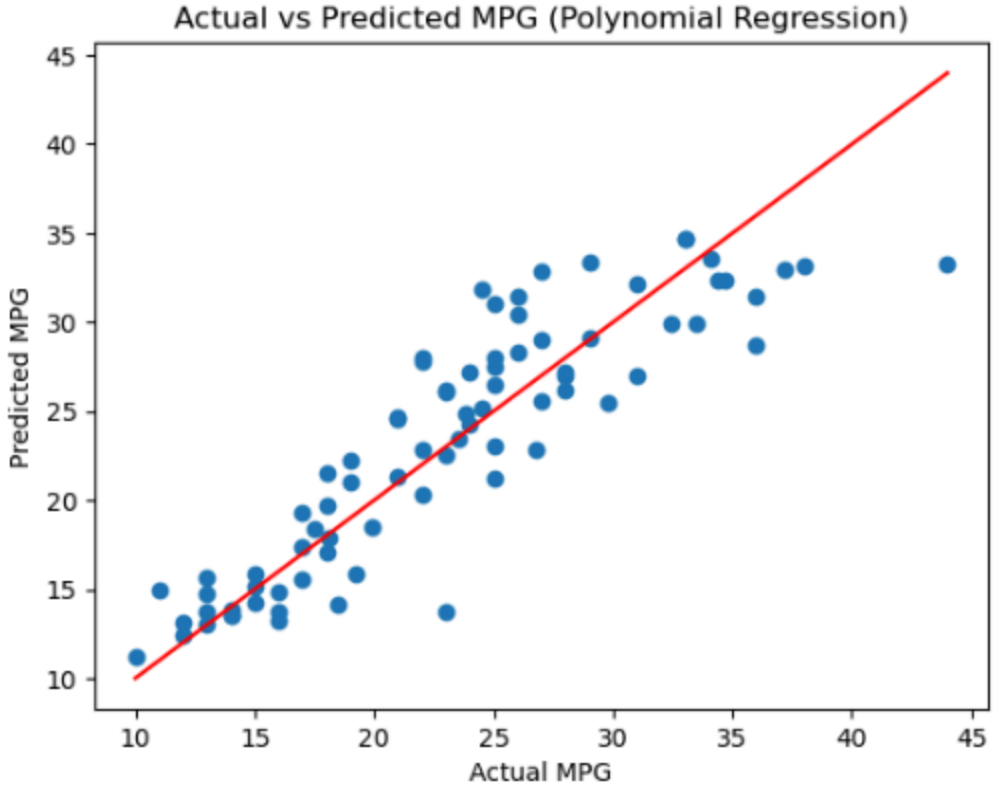
```
df = pd.read_csv(os.path.join(path, "auto-mpg.csv"))

df.head()

df.info()

df.isnull().sum()

df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    398 non-null    object
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
 6   model year    398 non-null    int64
 7   origin        398 non-null    int64
 8   car name      398 non-null    object
dtypes: float64(3), int64(4), object(2)
memory usage: 28.1+ KB
```

	mpg	cylinders	displacement	weight	acceleration	model year	origin
count	398.000000	398.000000	398.000000	398.000000	398.000000	398.000000	398.000000
mean	23.514573	5.454774	193.425879	2970.424623	15.568090	76.010050	1.572864
std	7.815984	1.701004	104.269838	846.841774	2.757689	3.697627	0.802055
min	9.000000	3.000000	68.000000	1613.000000	8.000000	70.000000	1.000000
25%	17.500000	4.000000	104.250000	2223.750000	13.825000	73.000000	1.000000
50%	23.000000	4.000000	148.500000	2803.500000	15.500000	76.000000	1.000000
75%	29.000000	8.000000	262.000000	3608.000000	17.175000	79.000000	2.000000
max	46.600000	8.000000	455.000000	5140.000000	24.800000	82.000000	3.000000

```
import kagglehub

import pandas as pd

import os

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

path = kagglehub.dataset_download("uciml/autompg-dataset")

df = pd.read_csv(os.path.join(path, "auto-mpg.csv"))

df['horsepower'] = pd.to_numeric(df['horsepower'], errors='coerce')

df.fillna(df.mean(numeric_only=True), inplace=True)

X = df[['displacement', 'horsepower', 'weight', 'acceleration']]

y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)

X_test_poly = poly.transform(X_test)

model = LinearRegression()

model.fit(X_train_poly, y_train)

y_pred = model.predict(X_test_poly)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

print("R2 Score:", r2_score(y_test, y_pred))

plt.scatter(y_test, y_pred)

plt.plot([y_test.min(), y_test.max()],

         [y_test.min(), y_test.max()],

         color='red')

plt.xlabel("Actual MPG")

plt.ylabel("Predicted MPG")

plt.title("Actual vs Predicted MPG (Polynomial Regression)")

plt.show()
```

OUTPUT:

```
Mean Squared Error: 11.133423286620843
R2 Score: 0.7929298925108685
```